



Dokumentácia riešenia interpretu pre OOP jazyk SOL26

Vysoké učení technické v Brně

Fakulta informačních technologií

Samuel Fačka (xfackas00)

Obsah

1. NÁVRH A ARCHITEKTÚRA INTERPRÉTU	3
1.1. VSTUP A PARSOVANIE	3
1.2. STATICKÁ VALIDÁCIA.....	3
1.3. RUNTIME OBJEKTOVÝ MODEL	3
1.4. VYKONÁVANIE	3
1.5. VSTAVANÝ SYSTÉM TRIED	4
2. INTERNÉ DÁTOVÉ ŠTRUKTÚRY	5
2.1. SoOBJECT	5
2.2. SoLCLASS	5
2.3. ENVIRONMENT	5
2.4. SUPERWRAPPER.....	5
2.5. AST MODEL	5
3. NÁVRHOVÉ VZORY A PRINCÍPY OOP	6
3.1. SINGLE RESPONSIBILITY PRINCIPLE	6
3.2. INTERPRETER PATTERN	6
3.3. CHAIN OF RESPONSIBILITY	6
4. PROBLÉMY A ICH RIEŠENIA.....	7
5. ROZŠÍRITEĽNOSŤ	8
5.1. MECHANISMUS DOES NOT UNDERSTAND	8
5.2. MODIFIKÁTORY PRÍSTUPU	8
6. VYUŽITIE AI	8

1. Návrh a architektúra interpretu

1.1. Vstup a parsovanie

Vstupný SOL-XML súbor je deserializovaný pomocou knižnice *pydantic-xml* priamo do stromu typovaných Pydantic modelov (*Program*, *ClassDef*, *Block*, *Assign*, *Expr*, ...). Výraz *Expr* je union štyroch vzájomne sa vylučujúcich variantov, pričom ich správnosť overuje Pydantic validátorom. Poradie priradení a parametrov v bloku je garantované atribútom *order*, podľa ktorého sa zoznamy zoradia okamžite po parsovaní.

1.2. Statická validácia

Pred samotným spustením *Interpreter* vykoná sedem sémantických kontrol nad AST. Patria sem napríklad: zákaz redefinície vstavaných tried, kontrola jedinečnosti názvov tried a selektorov, existencia metódy *Main.run*, zhoda arity selektora s aritou bloku, absencia duplicitných a kolíznych parametrov a detekcia cyklickej dedičnosti sledovaním reťaze rodičov.

1.3. Runtime objektový model

Triedy jazyka SOL26 sú za behu reprezentované inštanciami *SolClass* s dvoma tabuľkami metód – *methods* pre inštančné a *class_methods* pre triedne. Každý objekt je inštanciou *SolObject* so slovníkom *attributes*, ktorý slúži na viacero účelov. Ukladá číselnú či reťazcovú hodnotu pod kľúčom *__value__*, a pri blokových objektoch zároveň obsahuje celý uzáver, teda AST uzol bloku, zachytené lexikálne prostredie, *self* a aktuálnu triedu pre *super*. Typy *Nil*, *True* a *False* majú vždy jedinú zdieľanú inštanciu.

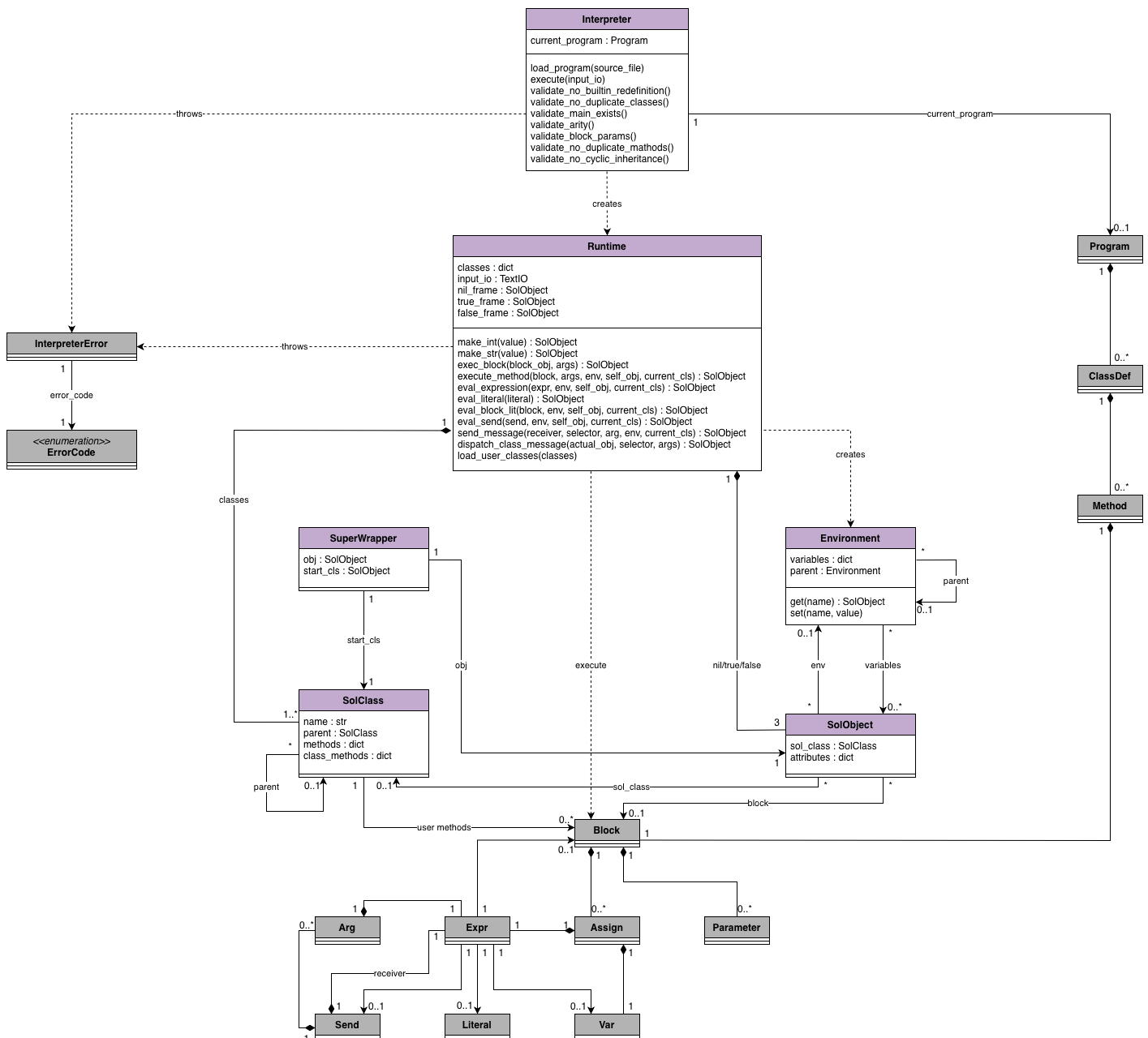
1.4. Vykonávanie

Interpretácia je realizovaná pomocou tree-walking, teda bez kompilácie či medzireprezentácie bajtkódom. Metóda *send_message()* rozlišuje tri prípady dispatchu: triednu správu, inštančnú metódu vyhľadajú smerom nahor po reťazi *parent*, a dynamický atribút inštancie, ak žiadna zodpovedajúca metóda nie je nájdená. Kľúčové slovo *super* je implementované pomocou triedy *SuperWrapper*, ktorý posúva začiatok vyhľadávania o jednu úroveň vyššie bez zmeny samotného objektu.

1.5. Vstavany systém tried

Vstavane metódy sú Python funkcie na úrovni modulu s jednotnou signatúrou (*receiver*, *args*, *runtime*), a tie sú uložené v *SolClass.methods*. Rovnaký slovník uchováva aj používateľsky definované metódy reprezentované AST uzlami typu *Block*. Dispatch v *send_message()* je preto jednotný: ak je nájdená hodnota volateľná, vykoná sa priamo, inak je odovzdaná metóde *execute_method()*.

1.6. UML diagram



UML diagram

2. Interné dátové štruktúry

2.1. SolObject

Základná runtime jednotka reprezentujúca každý objekt jazyka SOL26. Slovník *attributes* slúži na viacero účelov zároveň. Uchováva číselnú alebo reťazcovú hodnotu, dynamické atribúty inštancie, ako aj celý uzáver blokového objektu.

2.2. SolClass

Deskriptor triedy uchováajúci dve oddelené tabuľky metód: *methods* pre inštančné a *class_methods* pre triedne metódy. Hodnoty v *methods* môžu byť buď Python callable (vstavané metódy), alebo AST uzol *Block* (používateľsky definované metódy), čo umožňuje jednotný dispatch.

2.3. Environment

Lexikálny scope implementovaný ako reťaz rodič–dieťa, kde každé volanie metódy alebo bloku vytvára nový potomkovský scope. Vyhľadávanie premennej prebieha rekurzívne smerom nahor po tomto reťazci, pričom zápis aktualizuje existujúcu väzbu v najbližšom scope, v ktorom sa daná premenná nachádza.

2.4. SuperWrapper

Dočasný obal okolo *SolObject*, ktorý vzniká pri vyhodnotení kľúčového slova *super*. Uchováva referenciu na triedu, od ktorej sa má vyhľadávanie metódy posunúť o úroveň vyššie, bez zmeny samotného objektu prijímateľa.

2.5. AST model

Vstupný program je reprezentovaný stromom Pydantic modelov, ktorého zakoreňom je uzol *Program*, kde každý uzol zodpovedá konkrétnej syntaktickej konštrukcii SOL-XML. Počas celej interpretácie sa AST iba číta a nieje nijak modifikovaný.

3. Návrhové vzory a princípy OOP

3.1. Single Responsibility Principle

Zodpovednosti sú striktne rozdelené medzi dve triedy. *Interpreter* sa stará o načítanie SOL-XML a statickú validáciu, zataiľ čo *Runtime* zabezpečuje samotné vykonávanie programu. Toto oddelenie je viditeľné priamo v metóde *execute()*, kde sa inštancia *Runtime* vytvorí až po úspešnom prejdení všetkých siedmich validácií. Výhodou takéhoto návrhu je možnosť nezávislého testovania a rozširovania obo týchto súčastí.

3.2. Interpreter Pattern

Vykonávanie programu je realizované klasickým interpretom typu tree-walking, kde sa každý uzol AST spracúva rekurzívnym volaním príslušnej metódy. Vstupným bodom je *eval_expression()*, ktorá na základe typu uzla deleguje spracovanie na *eval_literal()*, *eval_block_lit()* alebo *eval_send()*. Metóda *eval_send()* následne vyhodnotí prijímateľa a argumenty a odovzdá riadenie metóde *send_message()*. Tento prístup nevyžaduje medzireprezentáciu ani kompiláciu, čo zjednodušuje implementáciu a ladenie.

3.3. Chain of Responsibility

Vyhľadávanie metód pri odoslaní správy je implementované ako prechod reťazou dedičnosti. Metóda *send_message()* iteruje od triedy prijímateľa smerom nahor cez reťaz *parent*, až kým nenájde požadovaný selektor alebo nedosiahne koniec hierarchie. Rovnaký mechanizmus funguje pre inštančné aj triedne správy. Vďaka tomu je pridanie novej triedy do hierarchie transparentné, teda postačuje korektne nastaviť *parent* bez akéhokoľvek zásahu v dispatchovacej logiky.

4. Problémy a ich riešenia

Jedným z hlavných implementačných problémov bolo správne fungovanie *super* v rámci uzáverov. Pri vytvorení bloku je potrebné zachytiť aktuálnu triedu (`__current_cls__`) do uzáveru, inak by *super* začal vyhľadávanie od runtime triedy objektu namiesto triedy kde bol blok definovaný. Táto informácia musí byť správne propagovaná cez volania:

`eval_block_lit()` -> `exec_block()` -> `execute_method()`.

Ďalším problémom bola kolízia medzi lexikálnou a runtime triedou pri dynamickom nastavovaní atribútov inštancie cez správu s jedným argumentom. V tomto prípade je potrebné kontrolovať kolíziu s existujúcimi metódami voči lexikálnej triede, teda triede, v ktorej bola metóda definovaná, nie voči runtime triede prijímateľa. Pri polymorfnom volaní by totiž runtime trieda mohla byť odlišná, čo by viedlo k nesprávnej kontrole.

Samostatným návrhovým problémom bolo zjednotenie dispatchu vstavaných a používateľsky definovaných metód. Vstavané metódy sú reprezentované ako Python callable, zatiaľ čo užívateľské metódy sú AST uzly typu *Block*, pričom oba typy sú uložené v rovnakom slovníku *methods*. Metóda `send_message()` ich rozlišuje jednou podmienkou: ak je hodnota volateľná, vykoná sa priamo, inak je odovzdaná metóde `execute_method()`.

5. Rozšíriteľnosť

5.1. Mechanismus `doesNotUnderstand`

Aktuálna implementácia v `send_message()` obsahuje prirodzené miesto kde toto rozšírenie zapadá. Keď vyhľadávanie metódy v reťazi dedičnosti zlyhá a nejde o dynamický atribút, aktuálne sa vyhodí chyba `INT_DNU`. Namiesto toho by stačilo pridať jeden fallback krok, ktorý zavolá metódu `doesNotUnderstand`: na prijímateľovi s novým objektom `Message`, zapuzdrujúcim selektor a argumenty. Samotná metóda by bola zaregistrovaná v `SolClass.methods` triedy `Object` ako vstavané callable, rovnakým spôsobom ako všetky ostatné vstavané metódy. Ochrana proti nekonečnej rekurzii by si vyžadovala jednoduchý flag na úrovni `Runtime`.

5.2. Modifikátory prístupu

`SolClass` uchováva metódy v slovníku `methods`. Na podporu modifikátorov prístupu by stačilo pridať druhý slovník `method_visibility`: `dict[str, str]` s hodnotami `public`, `protected` alebo `private`. Kontrola viditeľnosti by sa vykonávala priamo v `send_message()` pred samotným zavolaním metódy. Overovalo by sa napríklad, či prijímateľ zodpovedá `self` alebo `super` v prípade `protected`, teda či volanie neprichádza zvonku v prípade `private`. Dedičnosť viditeľnosti by prirodzene využívala existujúci mechanizmus prechodu reťazcom `parent`.

6. Využitie AI

Pri implementácii interpretu som využíval nástroj *ChatGPT*, prevažne pre dva účely. V prvom prípade išlo o objasnenie častí zadania, ktoré neboli jednoznačné a druhom prípade išlo o rôzne implementačné problémy.

Príslušné konverzácie sú dostupné na:

<https://chatgpt.com/share/69e8f28c-d09c-8329-975e-00ec19ac6eea>

<https://chatgpt.com/share/69e8f29b-c1e4-83eb-acc6-6bb634a2d197>